

TPM Alignment

Total Productive Maintenance · Engineering Alignment Document

Site Cinerglass
Lommel

Date 10 June
2026

Status Living
document

Repo
github.com/Explodingding/cinernet

Engineering Alignment Document — Cinerglass Lommel Plant

Document	docs/TPM_ALIGNMENT.md
Audience	Plant Engineering, Maintenance Management, Automation Engineering
Scope	Electrical distribution topology — Utility Building, Furnace 10, future wings
Status	Living document — updated alongside application releases

1. Executive Summary — An Engineering Tool for Slashing MTTR

CINERNET is a graph-based digital twin of the plant's electrical distribution network. Every transformer, MV switchgear panel, LV distribution board, field cabinet, and end load is modelled as a typed node; every power cable, control circuit, and signal run is modelled as a typed, directed edge. The result is a **live, queryable single-line diagram (SLD)** that replaces the static paper drawings (SMT-5250, SMT-5255, LAY-5246) currently used during fault response.

The MTTR problem

Mean Time To Repair decomposes into distinct phases, and in electrical troubleshooting the dominant cost is rarely the repair itself:

$$\text{MTTR} = T(\text{detect}) + T(\text{diagnose}) + T(\text{locate}) + T(\text{repair}) + T(\text{verify})$$

Historically 60–80 % of total downtime for
distribution faults: tracing the failure path
through paper schematics and physical walkdowns.

CINERNET attacks the diagnose and locate phases directly:

- T(diagnose):** the Fault Cascade engine (`lib/faultCascade.ts`) computes the complete set of downstream-affected equipment the instant a fault is registered — in milliseconds, at render time, with zero manual schematic tracing (see §5).
- T(locate):** every node carries a structured `physicalLocation` record (building, zone, floor, elevation, area, grid reference), so the technician knows not only *what* failed but *where it physically is* — basement –8 m, ground 0 m, or mezzanine +5 m — before leaving the control room.
- T(verify):** the same cascade map provides the re-energisation checklist in reverse: clear the root fault, and the derived statuses across the affected branch clear with it, confirming the full circuit is restored.

OEE linkage

In the OEE model (`Availability × Performance × Quality`), unplanned electrical downtime is a direct Availability loss. Every minute shaved off fault localisation is a minute returned to furnace operation — and for a continuous glass-melting process, where an uncontrolled power loss risks thermal damage to the furnace itself, fast and *correct* fault scoping is also a Quality and asset-protection mechanism, not merely a throughput one.

2. Pillar 1 — Autonomous Maintenance Support

TPM principle: operators perform first-line inspection and diagnosis on the equipment they run, reserving specialist engineering time for specialist problems.

The barrier to autonomous maintenance in electrical systems has always been **schematic literacy**. A furnace operator can see that cooling fan CAF10.1 has stopped — but determining *why* requires reading a riser diagram, understanding the panel naming convention, and knowing which upstream board feeds which field cabinet. That knowledge traditionally lives with one or two automation engineers.

The 3-Tier Progressive Disclosure system

CINERNET encodes that knowledge into the map itself, exposed through three depth tiers (`displayTier: 1 | 2 | 3`) in `types/topology.ts` , driven by the `activeTier` selector in the top bar:

Tier	Audience	Visible equipment	Question answered
1 — Site overview	Operators, shift leaders, management	MV feed, 35 kV switchgear, main transformers (TR1.1–TR1.3), primary LV/PFC panels	"Which building / main feed is affected?"
2 — Building detail	Line technicians	+ distribution cabinets, sub-panels (F1-MDP-1...9, F1-HOT-DP, ...)	"Which board do I open, and what feeds it?"
3 — Circuit detail	Electricians, automation engineers	+ individual loads: cooling pumps, CAF fans, batch drives, junction boxes	"Which breaker, which cable, which motor?"

The operational effect:

1. **A floor operator at Tier 1** sees instantly that the F10 wing is fed from TR1.1 via the cross-building tray run — without knowing what a PFC panel is. If the fault cascade shows the whole wing soft-red, the problem is upstream in the Utility Building and *no amount of local resetting will help*. That single insight prevents the most common autonomous-maintenance failure mode: operators power-cycling downstream equipment while the root cause sits two buildings away.
2. **A line technician at Tier 2** identifies the exact distribution cabinet in the chain and reads its `troubleshootingSteps` — a structured, per-device checklist (e.g. "Check main ACB Q0 (ON/OFF/TRIP) and PFC relay", "Measure busbar voltage: 400 V ±5 %") authored once by the responsible engineer and served to every technician thereafter.
3. **Tier 3 remains available** for the specialist, with terminal-box detail (`terminalBox`) data imported from commissioning CSVs) down to individual terminals.

Because the upstream/downstream relationships are *rendered* rather than *recalled*, the dependency knowledge of the automation engineer is democratised to every shift — which is precisely the staffing resilience TPM's autonomous maintenance pillar exists to create.

Physical-elevation layout as a training aid

The layout engine (`lib/siteLayout.ts`) maps physical elevation directly to screen position: mezzanine +5 m at the top of the canvas, ground 0 m in the middle, basement –8 m at the bottom, with power flowing visually upward from the

substation. A technician who has used CINERNET for a week has internalised the plant's vertical power architecture — the map doubles as an onboarding and cross-training instrument.

3. Pillar 2 — Planned & Preventive Maintenance

TPM principle: shift the maintenance mix from reactive to scheduled, condition-based work, planned around production windows.

What exists today

The `DetailDrawer` (right-hand panel, opened by tapping any node or edge) already provides the data substrate for planned maintenance:

- **Structured asset specifications** — `DeviceSpecs` (voltage, current, power, protection class, manufacturer) and `CableSpecs` (cross-section, max load, length, installation type) per element.
- **Per-device documentation** — `DocEntry` records linking commissioning drawings and test reports (SMT-5250 riser plan, LAY-5246 layout, panel location drawings) directly to the device card, served from `public/docs/`.
- **Change-log history** — `useElementHistory` (`lib/useElementHistory.ts`) records status transitions and free-text engineering notes per element, with author and timestamp (e.g. the TR-SPARE insulation-resistance investigation, ABB service ticket #ABB-2026-0421). This is the seed of a proper maintenance ledger: every intervention leaves a trace *on the asset that received it*, not in a disconnected spreadsheet.

Design concept: Asset Health panel

The next evolution of the `DetailDrawer` is a dedicated **Asset Health** section for the maintenance-critical asset classes — transformers (TR1.1/1.2/1.3, TR-SPARE) and main LV/PFC panels — structured as follows:

ASSET HEALTH			
Health index	● 87 / 100	(derived, see below)	
Thermal inspection			
Last thermographic survey	2026-03-14	PASS	
Next due	2026-09-14	[86 days]	
Hotspot record	L2 busbar joint, ΔT 4.2 K (within tol)		
Calibration & protection			
Protection relay calibration	due 2026-11-01		
ACB Q0 trip-test	due 2026-08-15		
PFC capacitor bank inspection	due 2026-07-01	⚠ 21 d	
Statutory			
Insulation resistance test	5-year cycle, 2028		

Implementation path (no schema upheaval required):

1. **Schema extension** — add an optional `maintenance` block to `TopologyNodeInput` : `{ inspections: InspectionRecord[]; nextDue: ISODate; healthIndex?: number }`. The existing rigid typing discipline (§6) guarantees that every entry is validated at compile time.
2. **Visual surfacing** — devices with an inspection due inside a configurable window (e.g. 30 days) receive a subtle badge on the map card, exactly as `circuitCount` badges render today. Overdue items escalate to the amber

`investigation` status — reusing the existing status pipeline rather than inventing a parallel one.

3. **Planning view** — because every node carries `physicalLocation` and building membership, due-soon items can be grouped into geographic work packages ("all thermal inspections in F10 electrical room, week 36"), minimising production interruption — the heart of *planned* maintenance.

Why this matters for the TPM audit

TPM planned-maintenance maturity is assessed on traceability: *can you show, per asset, what was done, when, by whom, and what is scheduled next?* CINERNET answers all four from a single tap on the asset, with the change log providing the historical axis and the Asset Health panel the forward axis.

4. Pillar 3 (supporting) — Training & Knowledge Capture

A brief but important note: the `troubleshootingSteps` arrays embedded in every node and edge constitute **institutionalised expert knowledge**. Each checklist is authored once by the engineer who knows the equipment best and is then available to every technician, on every shift, forever. When the expert retires or rotates, the checklist remains. This is TPM's training pillar implemented as data rather than as classroom sessions.

5. MTTR Reduction Mechanics — The Fault Cascade Engine

How fault localisation works today (without CINERNET)

1. Alarm or operator report: "fan stopped in F10 hot zone."
2. Technician retrieves the riser plan (SMT-5250), traces from the load upward through F1-HOT-DP, TR-DP1.x, the cross-building tray, TR1.x, to the 35 kV switchgear.
3. Physical walkdown of each candidate panel — basement, ground floor, mezzanine, two buildings.
4. Scope of impact is discovered *incrementally*, panel by panel.

Elapsed time before the first meaningful repair action: routinely 30–90 minutes.

How it works with CINERNET

The cascade engine (`lib/faultCascade.ts`) implements a **directed breadth-first search** over the topology graph, following edges in the power-flow direction (source → target):

```
// lib/faultCascade.ts
export type DerivedStatuses = Map<string, DerivedStatus>;
```

Mechanics, and why each property matters operationally:

Property	Implementation	Operational impact
Instant scope	BFS from every <code>fault</code> / <code>investigation</code> node over a prebuilt adjacency list; $O(V + E)$	The complete blast radius — every affected cabinet, cable, and load — is on screen the moment the fault is registered
Derived vs. confirmed	Downstream elements get <code>derived-fault</code> (soft red), distinct from the hard-red root	The technician immediately distinguishes <i>the</i> failure from <i>consequences</i> of the failure — no wasted truck-rolls to healthy-but-dark equipment
Severity precedence	<code>derived-fault</code> outranks <code>derived-investigation</code> when cascades overlap	Multiple simultaneous events present a correct, merged picture instead of flickering ambiguity
Edge illumination	Every cable on the affected path renders bright red with animated dash-march (<code>PowerCableEdge.tsx</code>)	The <i>route</i> of the failure is traceable by eye across buildings — the cross-building tray run from Utility to F10 lights up as a single continuous path
Computed, never stored	Derived statuses are calculated at render time and never written to the data model	The map cannot drift out of sync with reality; clearing the root fault clears the entire cascade atomically
Rehearsable	Demo presets ("MV Substation Fault", "F10 Cabinet Fault") inject faults via UI state	Fault-response drills can be run against the live model without touching real equipment — a TPM education tool in itself

The net effect on the MTTR decomposition: **T(diagnose) and T(locate) collapse from tens of minutes to seconds**, and T(verify) inherits a ready-made restoration checklist (the cascade, walked in reverse).

6. Software Poka-Yoke — Error-Proofing the Engineering Data

TPM / quality principle: design the process so the error cannot be made, rather than inspecting for it afterwards.

A digital twin is only as trustworthy as its data. CINERNET applies poka-yoke at the **data-entry and integration boundary**, using the TypeScript compiler itself as the inspection gate. Bad topology data does not produce a wrong map — it produces a **build failure**, before deployment.

Gate 1 — Rigid domain schemas (`types/topology.ts`)

Every property that drives rendering, layout, or cascade logic is a closed union type, not a free string:

- `AssetType` — exactly seven values (`mv-feed` ... `motor`). A typo like `"transformer"` is a compile error, not a silently unstyled node.
- `TopologyLayer` — the seven-level electrical hierarchy that drives vertical placement. An invalid layer cannot enter the layout engine.
- `BuildingId` / `LocationZone` — closed sets; assigning a node to a non-existent building is impossible.
- `Status` — exactly `operational | investigation | fault`. The cascade algorithm can never encounter an unknown state.
- `physicalLocation`, `specs`, `troubleshootingSteps` — **required** fields. A node cannot be added without location data and a troubleshooting checklist; the schema enforces the documentation standard that paper processes merely request.

Cross-cutting effect: when a contractor or future integrator adds Furnace 20 data, the compiler enforces the same completeness and vocabulary as the original dataset — consistency at scale without manual review of every entry.

Gate 2 — Centralised layout engine (`lib/siteLayout.ts`)

No node carries hand-typed canvas coordinates. Position is **derived** from declared physical facts:

- `LAYER_RANK` maps the electrical hierarchy to vertical order within a floor band — a transformer *cannot* be drawn below the loads it feeds.
- `FLOOR_BANDS` maps physical elevation to canvas Y — basement equipment *cannot* appear above ground-floor equipment.
- The two-pass tree-centering algorithm (leaf counting bottom-up, X assignment top-down) positions parents over their downstream subtrees automatically — branch symmetry is computed, not drawn.

The classic CAD failure mode — a schematic that is *electrically* correct but *spatially* misleading because someone dragged a symbol — is structurally impossible. The only escape hatch, `positionOverride`, is an explicit, named, reviewable declaration rather than an invisible drag.

Gate 3 — Render-time derivation (no stored derived state)

Cascade statuses, parallel-lane indices, and tier visibility are all computed from source data at render time. There is no cached "affected equipment list" that can go stale — eliminating an entire category of wrong-information incidents that plague manually maintained fault boards.

Gate 4 — Controlled vocabulary for integration identifiers

`externalRefs` (`scadaTag`, `osapiensAssetId`) gives every element typed, named slots for its SCADA and asset-management identities. Integration mapping errors surface as type errors at the boundary — see the companion document `SCALABILITY_ROADMAP.md` for the full integration lifecycle.

7. Summary — TPM Pillar Coverage Matrix

TPM pillar	CINERNET mechanism	Maturity
Autonomous maintenance	3-tier disclosure, per-device checklists, physical-elevation map	Live
Planned / preventive maintenance	Specs + docs + change-log per asset; Asset Health panel	Live foundation, panel designed (\$3)
Quality maintenance / poka-yoke	Closed-union schemas, derived layout, render-time computation	Live
Training & knowledge capture	Embedded troubleshooting steps, change-log notes, drill presets	Live
Focused improvement (Kobetsu Kaizen)	Fault history per element → recurring-failure analysis	Roadmap (requires telemetry, see <code>SCALABILITY_ROADMAP.md</code>)
Early equipment management	Schema-enforced data completeness for new installations (F20)	Architecture ready

CINERNET is not a documentation viewer; it is the plant's electrical knowledge, made executable. Each TPM pillar above maps to a concrete, inspectable code path in the repository — which is exactly the standard of evidence an engineering organisation should demand from its tooling.